

SQLite Connection

<code>sqlite3.connect(path)</code>	Open or create a database file	→ Connection
<code>conn.cursor()</code>	Create a cursor to run SQL	→ Cursor
<code>cursor.execute(sql, params?)</code>	Run one SQL statement	params: tuple
<code>cursor.executemany(sql, data)</code>	Run same SQL for many rows	data: list[tuple]
<code>conn.commit()</code>	Save INSERT / UPDATE / DELETE	after every write
<code>conn.close()</code>	Close connection when done	end of notebook
<code>cursor.fetchone()</code>	Get one row from results	→ tuple None
<code>cursor.fetchall()</code>	Get all rows from results	→ list[tuple]

SQL — SELECT / Filter / Sort

<code>SELECT *</code>	All columns	
<code>SELECT col1, col2</code>	Specific columns only	
<code>WHERE col = 'val'</code>	Exact match — also != > < >= <=	
<code>AND / OR</code>	Combine multiple conditions	
<code>LIKE 'A%'</code>	Starts with A (% = wildcard)	
<code>ORDER BY col ASC</code>	Sort A→Z / smallest first	or DESC
<code>LIMIT n</code>	Return at most n rows	int
<code>COUNT(*)</code>	Count rows — use in SELECT	
<code>GROUP BY col</code>	Aggregate per unique value	with COUNT/AVG/SUM
<code>JOIN t2 ON t1.id = t2.id</code>	Combine tables via foreign key	
<code>PRAGMA table_info(t)</code>	Column names + data types	

SQL — Write Operations

<code>INSERT INTO t (cols) VALUES (?)</code>	Add a new row — use ? placeholders	then commit()
<code>UPDATE t SET col=? WHERE ...</code>	Change existing data	always use WHERE
<code>DELETE FROM t WHERE ...</code>	Remove rows	always use WHERE
<code>INSERT OR IGNORE ...</code>	Skip if row already exists	safe for re-runs

pandas

<code>pd.read_sql(sql, conn)</code>	Run SQL → DataFrame	
<code>df.head(n)</code>	First n rows (default 5)	
<code>df[df['col'] == val]</code>	Boolean filter — equal	
<code>df[df['col'] > val]</code>	Boolean filter — greater than	
<code>df.sort_values('col')</code>	Sort DataFrame	ascending=True/ False
<code>pd.read_csv('file.csv')</code>	Load CSV file	
<code>df.to_sql(name, conn, ...)</code>	Write DataFrame to DB table	if_exists='append'

ipywidgets

<code>widgets.Text()</code>	Single-line text input	
<code>widgets Dropdown(options=[...])</code>	Select from a list	
<code>widgets.IntText()</code> / <code>FloatText()</code>	Number input	
<code>widgets.Button()</code>	Clickable button	<code>button_style='success'</code>
<code>widgets.Output()</code>	Area for printed messages	
<code>widgets.VBox([...])</code>	Stack widgets vertically	
<code>widget.value</code>	Read the current value	any type
<code>btn.on_click(fn)</code>	Call fn when button clicked	<code>fn(btn) -> None</code>
<code>output.clear_output()</code>	Wipe previous messages	inside with <code>output:</code>
<code>display(widget)</code>	Render widget in notebook	from <code>IPython.display</code>

matplotlib

<code>plt.figure(figsize=(w,h))</code>	New figure, size in inches	<code>tuple[int,int]</code>
<code>plt.bar(x, y)</code>	Vertical bar chart	
<code>plt.barh(x, y)</code>	Horizontal bar chart	
<code>plt.title('text')</code>	Chart title	required
<code>plt.xlabel / plt.ylabel</code>	Axis labels	required
<code>plt.xticks(rotation=45)</code>	Rotate x-axis labels	<code>int</code> degrees
<code>plt.tight_layout()</code>	Prevent label clipping	required
<code>plt.show()</code>	Display the chart	call last

Common Fixes

no such table	Run the setup cell first
nothing on click	Check <code>btn.on_click(fn) + display()</code>
changes not saved	Add <code>conn.commit()</code> after every write
cell won't run	Kernel → Restart & Run All

Worked Examples

PRAGMA — inspect a table's columns and types

```
cursor.execute("PRAGMA table_info(products)")
for row in cursor.fetchall():
    print(row)
# each row: (cid, column_name, type, notnull, default, pk)
```

COUNT — total rows in a table

```
cursor.execute("SELECT COUNT(*) FROM orders")
total = cursor.fetchone()[0]
print(f"Total orders: {total}")
```

read_sql — load a table into a DataFrame

```
df = pd.read_sql("SELECT * FROM countries", conn)
df.head() # first 5 rows
```

WHERE — filter rows in SQL

```
query = """
SELECT title, price
FROM books
WHERE price < 15
"""

pd.read_sql(query, conn)
```

Boolean filter — filter rows in pandas

```
df = pd.read_sql("SELECT * FROM animals", conn)
big = df[df['weight_kg'] > 100]
big
```

ORDER BY — sort results

```
pd.read_sql(
    "SELECT * FROM cities ORDER BY population DESC", conn
)
```

INSERT — add a new row

```
cursor.execute(
    "INSERT INTO items (label, quantity) VALUES (?, ?)",
    ("Widget A", 42)
)
conn.commit()
```

ipywidgets — a simple data entry form

```
# Populate a dropdown from the database
cats = pd.read_sql("SELECT cat_id, cat_name FROM categories", conn)
cat_opts = [(r["cat_name"], r["cat_id"]) for _, r in cats.iterrows()]

label_w = widgets.Text(description="Label:")
qty_w = widgets.IntText(description="Qty:")
cat_w = widgets.Dropdown(options=cat_opts, description="Category:")
btn = widgets.Button(description="Save", button_style="success")
out = widgets.Output()

def on_save(b):
    with out:
        out.clear_output()
        cursor.execute(
            "INSERT INTO items (label, qty, cat_id) VALUES (?, ?, ?)",
            (label_w.value, qty_w.value, cat_w.value))
        conn.commit()
        print(f"Saved: {label_w.value}")
        label_w.value = ""

btn.on_click(on_save)
display(widgets.VBox([label_w, qty_w, cat_w, btn, out]))
```

JOIN + GROUP BY — count rows across two tables

```
query = """
SELECT c.country_name, COUNT(*) AS total
FROM sales s
JOIN countries c ON s.country_id = c.country_id
GROUP BY c.country_name
"""

df_summary = pd.read_sql(query, conn)
```

bar chart — from a DataFrame column

```
plt.figure(figsize=(7, 4))
plt.bar(df_summary['country_name'], df_summary['total'], color='steelblue')
plt.title('Sales by Country')
plt.xlabel('Country')
plt.ylabel('Sales')
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```

read_csv + to_sql — import a CSV into the database

```
df_new = pd.read_csv('new_records.csv')
df_new.to_sql('items', conn, if_exists='append', index=False)
# Verify the import
pd.read_sql("SELECT COUNT(*) FROM items", conn)
```